

Compiladores 2026 — Fase 2

Guia de Pruebas: como demostrar cada feature del compilador

1. Como ejecutar las pruebas

1.1. Requisitos

- Python 3.10 o superior.
- Modulo PLY 3.11 o superior: `pip install ply`.
- Para la UI web: PHP y un navegador.

1.2. Modos de ejecucion

Tres formas de probar el compilador

1. **UI escritorio:** `python main.py` Editor + paneles de tokens, AST, semantico, TAC, errores. Atajo F5 ejecuta el analisis; boton "REPORTES HTML" genera los archivos en una carpeta a eleccion.
2. **UI web:** servir `index.php` con PHP. Ejecuta el codigo en el navegador y llama a `bridge.py`.
3. **CLI / JSON:** `python bridge.py archivo.programa` Imprime el resultado completo en JSON. Con la opcion `-reportes carpeta/` tambien genera los HTMLs.

1.3. Suites automaticas

```
python qa_rubrica.py      # 42 casos contra los 5 criterios de la rubrica
python qa_completo.py     # 90 casos exhaustivos (9 bloques A-I)
python qa_test.py         # 7 ejemplos precargados (bridge.py)
```

Estado actual: `qa_rubrica.py` 42/42 PASS, `qa_completo.py` 90/90 PASS, `qa_test.py` 7/7 PASS.

2. Sintaxis del lenguaje

Estructura:	<code>programa { ... }</code>
Tipos:	<code>entero decimal cadena booleano</code>
Control:	<code>si sino mientras hacer_mientras para</code>
I/O:	<code>imprimir leer</code>
Logicos:	<code>verdadero falso && !</code>

```
programa {
    entero edad = 25;
    decimal salario = 15750.50;
    cadena nombre = "Ana Garcia";
    booleano activo = verdadero;

    si (edad >= 18 && activo) {
        imprimir(nombre);
    } sino {
        imprimir("menor");
    }

    mientras (edad > 0) { edad = edad - 1; }
    para (entero i = 0; i < 5; i = i + 1) {
        imprimir(i);
    }
}
```

```
}  
}
```

3. Pruebas por criterio de la rubrica

3.1. Criterio 1: Deteccion de errores semanticos

3.1.1. P1.1 — Variable no declarada

```
programa { imprimir(noexiste); }
```

[Semantico] Linea 1, Col 20: Variable 'noexiste' no declarada

3.1.2. P1.2 — Variable duplicada

```
programa {  
    entero x = 1;  
    entero x = 2;  
}
```

[Semantico] Linea 3: Variable 'x' ya fue declarada en este ambito (linea 2)

3.1.3. P1.3 — Tipo incompatible en declaracion

```
programa {  
    entero x = "hola";  
    booleano b = 5;  
    entero y = 3.14;  
}
```

[Semantico] Linea 2: Asignacion incompatible: variable 'x' de tipo 'entero' no puede recibir un valor 'cadena'
[Semantico] Linea 3: Asignacion incompatible: variable 'b' de tipo 'booleano' no puede recibir un valor 'entero'
[Semantico] Linea 4: Asignacion incompatible: variable 'y' de tipo 'entero' no puede recibir un valor 'decimal'

3.1.4. P1.4 — Widening permitido (no es error)

```
programa {  
    decimal x = 5;  
    decimal y = 2 + 3;  
}
```

Resultado esperado

Sin errores. La promocion entero → decimal es semanticamente segura y se acepta.

3.1.5. P1.5 — Condicion no booleana

```
programa {  
    entero x = 5;  
    si (x) { imprimir(x); }  
    mientras (x + 1) { x = x - 1; }  
    para (entero i = 0; i; i = i + 1) { imprimir(i); }  
}
```

[Semantico] Linea 3: La condicion del 'si' debe ser booleana, no 'entero'
[Semantico] Linea 4: La condicion del 'mientras' debe ser booleana, no 'entero'
[Semantico] Linea 5: La condicion del 'para' debe ser booleana, no 'entero'

3.1.6. P1.6 — Division por cero

```
programa {  
    entero a = 10 / 0;  
    entero b = 10 % 0;  
}
```

[Semantico] Linea 2: Division por cero (operador '/')
[Semantico] Linea 3: Division por cero (operador '%')

3.1.7. P1.7 — Operacion aritmetica incompatible

```
programa {  
    booleano b = verdadero;  
    entero x = b + 1;  
}
```

[Semantico] Linea 3: Operacion aritmetica '+' incompatible entre
'booleano' y 'entero'

3.1.8. P1.8 — Comparacion incompatible

```
programa {  
    entero x = 5;  
    booleano b = x == "hola";  
}
```

[Semantico] Linea 3: Comparacion '==' incompatible entre 'entero' y 'cadena'

3.1.9. P1.9 — Operacion logica entre no booleanos

```
programa {  
    entero a = 1; entero b = 2;  
    booleano c = a && b;  
}
```

[Semantico] Linea 3: Operacion logica '&&' incompatible entre 'entero' y 'entero',

3.1.10. P1.10 — Operador unario invalido

```
programa {  
    booleano b = verdadero;  
    entero x = -b;  
    entero y = 5;  
    booleano c = !y;  
}
```

[Semantico] Linea 3: Operador unario '-' incompatible con 'booleano'
[Semantico] Linea 5: Operador '!' incompatible con 'entero'

3.1.11. P1.11 — Lectura de variable no declarada

```
programa { leer(variable_inexistente); }
```

[Semantico] Linea 1, Col 17: Variable 'variable_inexistente' no declarada

3.1.12. P1.12 — Asignacion a variable no declarada

```
programa { y = 10; }
```

[Semantico] Linea 1, Col 12: Variable 'y' no declarada

3.1.13. P1.13 — Uso antes de declaracion

```
programa {  
    imprimir(x);  
    entero x = 5;  
}
```

[Semantico] Linea 2: Variable 'x' usada antes de ser declarada
(declarada en linea 3)

3.2. Criterio 2: Reporte HTML semantico

3.2.1. P2.1 — Desde la UI Tkinter

1. Abrir python main.py.
2. Cargar el ejemplo "05 Errores Lexicos".
3. Presionar F5 para analizar.
4. Click en el boton **REPORTES HTML** (footer).
5. Elegir carpeta destino. Se generan 6 archivos.
6. Se abre automaticamente el reporte de errores semanticos en el navegador.

Lo que el reporte HTML muestra

- Encabezado con titulo y timestamp.
- Resumen con chips: total + cuenta por categoria (variables no declaradas, tipos incompatibles, division por cero, condiciones no booleanas, etc.).
- Tabla con columnas: #, tipo (tag morado), **linea**, **columna**, identificador afectado, descripcion del error.

3.2.2. P2.2 — Desde la CLI

```
python bridge.py mi_programa.programa --reportes ./salida_html/
```

Genera 6 archivos en salida_html/:

- reporte_errores_semanticos.html (el clave)
- reporte_errores.html
- reporte_tokens.html
- reporte_tabla_simbolos.html
- reporte_tac.html
- reporte_tac_optimizado.html

3.3. Criterio 3:Codigo de Tres Direcciones

3.3.1. P3.1 — Expresion aritmetica

```
programa {  
    entero a = 3;  
    entero b = 4;  
    entero c = (a + b) * 2;  
}
```

```
1: a = 3  
2: b = 4  
3: $t1 = a + b  
4: $t2 = $t1 * 2  
5: c = $t2
```

3.3.2. P3.2 — Si/sino

```
programa {  
    entero x = 5;  
    si (x > 0) { imprimir(x); } sino { imprimir(0); }  
}
```

```
1: x = 5  
2: $t1 = x > 0  
3: ifFalse $t1 goto $L1  
4: imprimir x  
5: goto $L2  
6: $L1:  
7: imprimir 0  
8: $L2:
```

3.3.3. P3.3 — Mientras

```
programa {  
    entero i = 0;  
    mientras (i < 10) { i = i + 1; }  
}
```

```
1: i = 0  
2: $L1:  
3: $t1 = i < 10  
4: ifFalse $t1 goto $L2  
5: $t2 = i + 1  
6: i = $t2  
7: goto $L1  
8: $L2:
```

3.3.4. P3.4 — Para y hacer-mientras

Para genera: init, label, comparacion, ifFalse, cuerpo, update, goto, label final.

Hacer-mientras pone la condicion al final, asi el cuerpo siempre se ejecuta al menos una vez.

3.4. Criterio 4: Optimizacion

3.4.1. P4.1 — Constant Folding

```
programa { entero c = (3 + 4) * 2 - 1; imprimir(c); }
```

Antes:

```
$t1 = 3 + 4
$t2 = $t1 * 2
$t3 = $t2 - 1
c = $t3
imprimir c
```

Despues (reduccion 60 %):

```
c = 13
imprimir 13
```

3.4.2. P4.2 — Identidades algebraicas

$a + 0 \rightarrow a$; $b * 1 \rightarrow b$; $c * 0 \rightarrow 0$.

3.4.3. P4.3 — Branch Pruning

```
programa { si (verdadero) { imprimir(1); } sino { imprimir(2); } }
```

La condicion siempre verdadera elimina el `ifFalse`. La rama `sino` queda inalcanzable y la pasada de *Unreachable Code Elimination* la elimina.

3.4.4. P4.4 — Punto fijo

El optimizador aplica las 6 pasadas en orden y repite el ciclo hasta que ninguna haga cambios:

1. Constant Folding & Algebraic
2. Constant / Copy Propagation
3. Branch Pruning
4. Unreachable Code Elimination
5. Dead-Code Elimination
6. Jump Threading

3.5. Criterio 5: Funcionamiento general

3.5.1. P5.1 — Los 7 ejemplos precargados

En la UI Tkinter (menu *Ejemplos*):

1. 01 Variables y Tipos
2. 02 Control de Flujo
3. 03 Bucles mientras y para
4. 04 Expresiones Complejas
5. 05 Errores Lexicos (a proposito)
6. 06 Strings y Booleanos
7. 07 Programa Completo

3.5.2. P5.2 — Pipeline completo via CLI

```
echo "programa { entero x = 5; imprimir(x); }" > test.programa
python bridge.py test.programa
```

JSON con claves: `tokens`, `simbolos`, `errores`, `errores_estructurados`, `tac`, `tac_optimizado`, `metricas`, `traza_optimizacion`, `reportes`.

3.5.3. P5.3 — UI web

```
php -S localhost:8000
# abrir http://localhost:8000/index.php
```

4. Casos limite y robustez

4.1. Programa vacio

```
programa { }
```

Resultado: 0 errores, TAC vacio.

4.2. Solo comentarios

```
// solo un comentario
/* y otro */
```

Resultado: 0 errores.

4.3. Programa con 50 declaraciones

50 variables `entero v0...v49`. Tabla de simbolos con 50 entradas. TAC con 50 cuádruplos. Analisis < 100 ms.

4.4. Anidacion profunda

```
programa { entero x = ((((((1 + 2) * 3) - 4) / 5) % 6) + 7); }
```

6 niveles de parentesis, parser sin problemas.

4.5. Cadenas con escapes

```
programa {
    cadena a = "linea1\nlinea2";
    cadena b = "comilla \"interna\"";
    cadena c = "";
}
```

5. Lista de chequeo para la demo

Checklist sugerido al presentar

1. `python main.py`.
2. Cargar "*01 Variables y Tipos*" → F5 → mostrar tokens, AST, simbolos, TAC.
3. Cargar "*05 Errores Lexicos*" → F5 → mostrar consola de errores con linea y columna.
4. Click **REPORTES HTML** → se abre el reporte semantico en el navegador (criterio 2).
5. Cargar "*07 Programa Completo*" → F5 → tab *Codigo Intermedio*: TAC original vs optimizado, metricas, traza.
6. Terminal: `python qa_rubrica.py` → 42/42 PASS.
7. Terminal: `python qa_completo.py` → 90/90 PASS.

6. Resumen de cobertura

Suite	Casos	Estado
qa_rubrica.py	42	42/42 PASS (100 %)
qa_completo.py	90	90/90 PASS (100 %)
qa_test.py	7	7/7 PASS (100 %)
TOTAL	139	139/139 PASS

Desglose de qa_completo.py por bloque:

Bloque	Area	Casos
A	Lexico	10/10
B	Sintactico	10/10
C	Semantico	25/25
D	TAC	17/17
E	Optimizacion	10/10
F	Reportes HTML	7/7
G	Pipeline	2/2
H	Stress	7/7
I	Regresion	2/2